

Algoritmos e Estruturas de Dados I (CCF 211)

Medida do Tempo de Execução de um Programa (Cap01 – Seção 1.3- Ziviani)

Complexidade Computacional

- O projeto de algoritmos é fortemente influenciado pelo estudo de seu comportamento
  - Um algoritmo não é “bom” “apenas” porque está correto!

- Depois que um problema é analisado e decisões de projeto são finalizadas, é necessário estudar as várias opções de algoritmos a serem utilizados, considerando os aspectos de **tempo** de execução e **espaço** ocupado
  - Análise da complexidade ou custo computacional

Tipos de Problemas na Análise de Algoritmos

- **Análise de um algoritmo em particular**.
  - Qual é o custo de usar um dado algoritmo para resolver um problema específico?
  - Características que devem ser investigadas:
    - análise do número de vezes que cada parte do algoritmo deve ser executada (**processamento**)
    - estudo da quantidade de **memória** necessária

Tipos de Problemas na Análise de Algoritmos

- **Análise de uma classe de algoritmos**.
  - Qual é o algoritmo de menor custo possível para resolver um problema particular?
  - Toda uma família de algoritmos é investigada.
  - Procura-se identificar um que seja o melhor possível.
  - Coloca-se **limites** para a complexidade computacional dos algoritmos pertencentes à classe.

Custo Ótimo de um Algoritmo

- Determinando o menor custo possível para resolver problemas de uma dada classe, temos a medida da dificuldade inerente para resolver o problema.

- Quando o custo de um algoritmo é igual ao menor custo possível, o algoritmo é **ótimo** para a medida de custo considerada.

Medida do Custo pela Execução do Programa (Tempo de Relógio)

- Medidas são bastante inadequadas e os resultados jamais devem ser generalizados:
  - resultados são dependentes do compilador que pode favorecer certas construções em detrimento de outras;
  - resultados dependem do *hardware*;
  - quando grandes quantidades de memória são utilizadas, as medidas de tempo podem depender deste aspecto.

Medida do Custo pela Execução do Programa (Tempo de Relógio)

- Apesar disso, há argumentos a favor de se obterem medidas reais de tempo.
  - Ex.: quando há vários algoritmos distintos para resolver um mesmo tipo de problema, todos com um custo de execução dentro de uma mesma ordem de grandeza.
  - Assim, são considerados tanto os custos reais das operações como os custos não aparentes, tais como alocação de memória, indexação, carga, dentre outros.

Medida do Custo por meio de um Modelo Matemático

- Modelo matemático baseado em computador idealizado
- O código do algoritmo é inspecionado, buscando-se identificar apenas a **operação mais significativa**
  - Ex.: algoritmos de ordenação. Consideramos o número de comparações entre os elementos do conjunto a ser ordenado e ignoramos as operações aritméticas, de atribuição e manipulações de índices, caso existam.
- Contabilizamos o número de vezes que a operação mais significativa é executada

Função de Complexidade

- Para medir o custo de execução de um algoritmo é comum definir uma função de custo ou **função de complexidade** *f*.
- *f(n)* é a medida do tempo necessário para executar um algoritmo para um problema de tamanho *n* (tamanho da entrada).
- Função de **complexidade de tempo**: *f(n)* mede o tempo necessário para executar um algoritmo em um problema de tamanho n.
- Função de **complexidade de espaço**: *f(n)* mede a memória necessária para executar um algoritmo em um problema de tamanho n.
- Utilizaremos *f* para denotar uma função de complexidade de tempo daqui para a frente.
- A complexidade de tempo na realidade não representa tempo diretamente, mas o número de vezes que determinada operação considerada relevante é executada.

Exemplo: maior elemento

- Considere o algoritmo para encontrar o maior elemento de um vetor de inteiros A[n]; n ≥ 1.

```
int Max(int* A, int n) {
   int i, Temp;

   Temp = A[0];
   for (i = 1; i < n; i++)
      if (Temp < A[i])
         Temp = A[i];
   return Temp;
}
```

- Seja *f* uma função de complexidade tal que *f(n)* é o número de comparações entre os elementos de A, se A contiver n elementos.
- **Qual é a função f(n)?**

Exemplo: maior elemento

- Considere o algoritmo para encontrar o maior elemento de um vetor de inteiros A[n]; n ≥ 1.

```
int Max(int* A, int n) {
   int i, Temp;

   Temp = A[0];
   for (i = 1; i < n; i++)
      if (Temp < A[i])
        Temp = A[i];
   return Temp;
}
```

$$f(n) = \sum_{i=1}^{n-1} 1 = n - 1$$

- Seja *f* uma função de complexidade tal que *f(n)* é o número de comparações entre os elementos de A, se A contiver n elementos. **Logo *f(n) = n -1***

Tamanho da Entrada de Dados

- Para alguns algoritmos, o custo de execução é uma função da entrada particular dos dados, não apenas do tamanho da entrada.

- No caso da função Max do programa do exemplo, o custo é uniforme sobre todos os problemas de tamanho n.

- Já para um algoritmo de ordenação isso pode não ocorrer: se os dados de entrada já estiverem quase ordenados, então o algoritmo pode ter que trabalhar menos.

Melhor Caso, Pior Caso e Caso Médio

- **Melhor caso**: menor tempo de execução sobre todas as entradas de tamanho n.

- **Pior caso**: maior tempo de execução sobre todas as entradas de tamanho n.
  - Se f é uma função de complexidade baseada na análise de pior caso, o custo de aplicar o algoritmo nunca é maior do que f(n).

- **Caso médio** (ou caso esperado): média dos tempos de execução de todas as entradas de tamanho n.

Análise de Melhor Caso, Pior Caso e Caso Médio

- Na análise do caso médio, supõe-se uma **distribuição de probabilidades** sobre o conjunto de entradas de tamanho n e o custo médio é obtido com base nessa distribuição.
- A análise do caso médio é geralmente muito mais difícil de obter do que as análises do melhor e do pior caso.
- É comum supor uma distribuição de probabilidades em que todas as entradas possíveis são igualmente prováveis.
- Na prática isso nem sempre é verdade.

Exemplo - Registros de um Arquivo

- Considere o problema de acessar os **registros** de um arquivo.
- Cada registro contém uma **chave** única que é utilizada para recuperar registros do arquivo.
- O problema: dada uma chave qualquer, localize o registro que contenha esta chave.
- O algoritmo de pesquisa mais simples é o que faz a **pesquisa sequencial**.

Exemplo - Registros de um Arquivo

- Seja f uma função de complexidade tal que *f(n)* é o número de registros consultados no arquivo (número de vezes que a chave de consulta é comparada com a chave de cada registro).
  - **melhor caso:**
  - **pior caso:**
  - **caso médio:**

Exemplo - Registros de um Arquivo

- Seja f uma função de complexidade tal que *f(n)* é o número de registros consultados no arquivo (número de vezes que a chave de consulta é comparada com a chave de cada registro).
  - **melhor caso:**
    - registro procurado é o primeiro consultado
  - **pior caso:**
  - **caso médio:**

Exemplo - Registros de um Arquivo

- Seja f uma função de complexidade tal que f(n) é o número de registros consultados no arquivo (número de vezes que a chave de consulta é comparada com a chave de cada registro).
  - **melhor caso:**
    - registro procurado é o primeiro consultado
    - f(n) = 1
  - **pior caso:**
  - **caso médio:**

Exemplo - Registros de um Arquivo

- Seja f uma função de complexidade tal que f(n) é o número de registros consultados no arquivo (número de vezes que a chave de consulta é comparada com a chave de cada registro).
  - **melhor caso:**
    - registro procurado é o primeiro consultado
    - f(n) = 1
  - **pior caso:**
    - registro procurado é último consultado ou não está presente;
  - **caso médio:**

Exemplo - Registros de um Arquivo

- Seja f uma função de complexidade tal que *f(n)* é o número de registros consultados no arquivo (número de vezes que a chave de consulta é comparada com a chave de cada registro).
  - **melhor caso:**
    - registro procurado é o primeiro consultado
    - f(n) = 1
  - **pior caso:**
    - registro procurado é último consultado ou não está presente;
    - f(n) = n
  - **caso médio:**

Exemplo - Registros de um Arquivo

- No estudo do caso médio, vamos considerar que toda pesquisa recupera um registro.
- Se p_i for a probabilidade de que o i-ésimo registro seja procurado, e considerando que para recuperar o i-ésimo registro são necessárias *i* comparações, então:

$$f(n) = 1 \times p_1 + 2 \times p_2 + 3 \times p_3 + \ldots + n \times p_n$$

Exemplo - Registros de um Arquivo

- Para calcular f(n) basta conhecer a distribuição de probabilidades p_i.
- Se cada registro tiver a mesma probabilidade de ser acessado que todos os outros, então

$$p_i = 1/n,\ 1 \leq i \leq n$$

Exemplo - Registros de um Arquivo

- Para calcular f(n) basta conhecer a distribuição de probabilidades p_i.
- Se cada registro tiver a mesma probabilidade de ser acessado que todos os outros, então

$$p_i = 1/n,\ 1 \leq i \leq n$$

- Nesse caso:

$$f(n) = \frac{1}{n}(1+2+3+\cdots+n) = \frac{1}{n}\left(\frac{n(n+1)}{2}\right) = \frac{n+1}{2}.$$

- A análise do caso esperado revela que uma pesquisa com sucesso examina aproximadamente metade dos registros.

Exemplo - Registros de um Arquivo

- Seja *f* uma função de complexidade tal que *f(n)* é o número de registros consultados no arquivo (número de vezes que a chave de consulta é comparada com a chave de cada registro).
  - **melhor caso:**
    - registro procurado é o primeiro consultado
    - $f(n) = 1$
  - **pior caso:**
    - registro procurado é o último consultado ou não está presente no arquivo;
    - $f(n) = n$
  - **caso médio:**
    - $f(n) = (n + 1)/2$.

Exemplo - Maior e Menor Elemento (1)

- Considere o problema de encontrar o maior e o menor elemento de um vetor de inteiros A[n]; n ≥ 1.
- Um algoritmo simples pode ser derivado do algoritmo apresentado no programa para achar o maior elemento.

```
void MaxMin1(int* A, int n, int* pMax, int* pMin)
{
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Qual a função de complexidade para MaxMin1?

```
void MaxMin1(int* A, int n, int* pMax, int* pMin)
{
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      if (A[i] < *pMin) *pMin = A[i];
   }
}
```

2*(n-1)

$$f(n) = \sum_{i=1}^{n-1} 2 = 2 * \sum_{i=1}^{n-1} 1 = 2 * (n-1)$$

Qual a função de complexidade para MaxMin1?

```
void MaxMin1(int* A, int n, int* pMax, int* pMin)
{
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      if (A[i] < *pMin) *pMin = A[i];
   }
}
```

- Seja *f(n)* o número de comparações entre os elementos de A, se A contiver *n* elementos.
- Logo ***f(n) = 2(n-1)*** para *n > 0*, para o melhor caso, pior caso e caso médio.

Exemplo - Maior e Menor Elemento (2)

- MaxMin1 pode ser facilmente melhorado: a comparação A[i] < *pMin só é necessária quando a comparação A[i] > *pMax dá falso.
  - Se A[i] já é maior do que alguém (pMax), não tem como ele ser o menor elemento do vetor

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

Pior caso:

Caso médio:

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

- quando os elementos estão em ordem crescente; Neste caso, (A[i] > *pMax) é sempre verdadeiro, entrando apenas no 1º if sempre.

Pior caso:

Caso médio:

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

- quando os elementos estão em ordem crescente; $f(n) = \sum_{i=1}^{n-1} 1 = n-1$

Pior caso:

Caso médio:

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

- quando os elementos estão em ordem crescente; $f(n) = \sum_{i=1}^{n-1} 1 = n-1$

Pior caso:

- quando os elementos estão em ordem decrescente; Neste caso, (A[i] > *pMax) é sempre falso, forçando a avaliação do 2º if sempre, que será sempre verdadeiro. Pior caso também ocorre quando todos os itens são iguais.

Caso médio:

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

- quando os elementos estão em ordem crescente; $f(n) = \sum_{i=1}^{n-1} 1 = n-1$

Pior caso:

- quando os elementos estão em ordem decrescente; $f(n) = 2\sum_{i=1}^{n-1} 1 = 2(n-1)$

Caso médio:

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

- quando os elementos estão em ordem crescente; $f(n) = \sum_{i=1}^{n-1} 1 = n-1$

Pior caso:

- quando os elementos estão em ordem decrescente; $f(n) = 2\sum_{i=1}^{n-1} 1 = 2(n-1)$

Caso médio:

- No caso médio, A[i] é maior do que pMax a metade das vezes.

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

- quando os elementos estão em ordem crescente; $f(n) = \sum_{i=1}^{n-1} 1 = n-1$

Pior caso:

- quando os elementos estão em ordem decrescente; $f(n) = 2\sum_{i=1}^{n-1} 1 = 2(n-1)$

Caso médio:

- No caso médio, A[i] é maior do que pMax a metade das vezes.
- ***f(n) = n – 1 + (n – 1)/2 = 3n/2 – 3/2***

Qual a função de complexidade para MaxMin2?

```
void MaxMin2(int* A, int n, int* pMax, int* pMin) {
   int i;

   *pMax = A[0];
   *pMin = A[0];
   for (i = 1; i < n; i++) {
      if (A[i] > *pMax) *pMax = A[i];
      else if (A[i] < *pMin) *pMin = A[i];
   }
}
```

Melhor caso:

- quando os elementos estão em ordem crescente; $f(n) = \sum_{i=1}^{n-1} 1 = n-1$

•Pior caso:

- quando os elementos estão em ordem decrescente; $f(n) = 2\sum_{i=1}^{n-1} 1 = 2(n-1)$

Caso médio:

- No caso médio, A[i] é maior do que pMax a metade das vezes.
- ***f(n) = 3n/2 – 3/2***

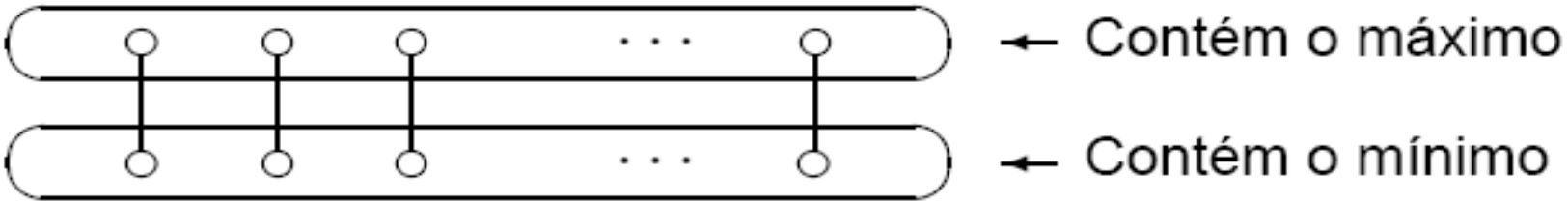
Exemplo - Maior e Menor Elemento (3)

• Considerando o número de comparações realizadas, existe a possibilidade de obter um algoritmo mais eficiente:

Compare os elementos de A aos pares, separando-os em dois subconjuntos (maiores em um e menores em outro), a um custo de $\lceil (n/2) \rceil$ comparações.

• O máximo é obtido do subconjunto que contém os maiores elementos, a um custo de $\lceil (n/2) \rceil$-1 comparações

• O mínimo é obtido do subconjunto que contém os menores elementos, a um custo de $\lceil (n/2) \rceil$-1 comparações



Qual a função de complexidade para este novo algoritmo?

- Os elementos de A são comparados dois a dois. Os elementos maiores são comparados com *p*Max* e os elementos menores são comparados com *p*Min*.
- Quando n é ímpar, o elemento que está na posição A[n-1] é duplicado na posição A[n] para evitar um tratamento de exceção.
- Para esta implementação:

$$f(n) = \frac{n}{2} + \frac{n-2}{2} + \frac{n-2}{2} = \frac{3n}{2} - 2,$$

no pior caso, melhor caso e caso médio

Exemplo - Maior e Menor Elemento (3)

```
void MaxMin3(int* A, int n, int* pMax, int* pMin) {
   int i, FimDoAnel;

   if ((n % 2) > 0)
   {  A[n] = A[n - 1]; FimDoAnel = n; }
   else FimDoAnel = n - 1;

   if (A[0] > A[1]) { *pMax = A[0]; *pMin = A[1]; }
   else             { *pMax = A[1]; *pMin = A[0]; }

   for(i=3;i<=FimDoAnel;i+=2)
   {
      if (A[i - 1] > A[i])
      {
         if (A[i - 1] > *pMax) *pMax = A[i - 1];
         if (A[i]     < *pMin) *pMin = A[i];
      }
      else
      {
         if (A[i - 1] < *pMin) *pMin = A[i - 1];
         if (A[i]     > *pMax) *pMax = A[i];
      }
   }
}
```

Comparação 1

Comparação 2

Comparação 3

Comparação 4

Comparação 3

Comparação 4

Qual a função de complexidade para MaxMin3?

- Quantas comparações são feitas em MaxMin3?

Qual a função de complexidade para MaxMin3?

- Quantas comparações são feitas em MaxMin3?
  - 1ª. comparação feita 1 vez
  - 2ª. comparação feita n/2 – 1 vezes
  - 3ª. e 4ª. comparações feitas n/2 – 1 vezes
  - n é a quantidade de elementos no vetor e será sempre par
  - 2ª, 3ª e 4ª comparações: não realizam com o 1º par de valores, já que isso é feito na 1ª comparação

Qual a função de complexidade para MaxMin3?

- Quantas comparações são feitas em MaxMin3?
  - 1ª. comparação feita 1 vez
  - 2ª. comparação feita n/2 - 1 vezes
  - 3ª. e 4ª. comparações feitas n/2 – 1 vezes (cada)

$f(n) = 1 + n/2 - 1 + 2 * (n/2 - 1)$

$f(n) = (3n - 6)/2 + 1$

$f(n) = 3n/2 - 3 + 1 = 3n/2 - 2$

Comparação entre os Algoritmos

- A tabela apresenta uma comparação entre os algoritmos dos programas MaxMin1, MaxMin2 e MaxMin3, considerando o número de comparações como medida de complexidade.
- Os algoritmos MaxMin2 e MaxMin3 são superiores ao algoritmo MaxMin1 de forma geral.
- O algoritmo MaxMin3 é superior ao algoritmo MaxMin2 com relação ao pior caso e bastante próximo quanto ao caso médio.

Os três algoritmos	$f(n)$		
	Melhor caso	Pior caso	Caso médio
MaxMin1	$2(n-1)$	$2(n-1)$	$2(n-1)$
MaxMin2	$n-1$	$2(n-1)$	$3n/2-3/2$
MaxMin3	$3n/2-2$	$3n/2-2$	$3n/2-2$